

AWS Certified DevOps Engineer - Professional (DOP-C02) Study Guide

Exam Code: DOP-C02

Duration: 180 minutes

Questions: 75 (65 scored + 10 unscored)

Passing Score: 750/1000

Format: Multiple choice, Multiple response

Recommended Experience: 2+ years provisioning, operating, and managing AWS environments

Exam Domain Weightings

Domain	Weight	Focus Area
Domain 1: SDLC Automation	22%	CI/CD, testing, deployment strategies
Domain 2: Configuration Management and IaC	17%	CloudFormation, CDK, Systems Manager
Domain 3: Resilient Cloud Solutions	15%	High availability, scalability, DR
Domain 4: Monitoring and Logging	15%	CloudWatch, X-Ray, observability
Domain 5: Incident and Event Response	14%	EventBridge, troubleshooting, automation
Domain 6: Security and Compliance	17%	IAM, encryption, auditing, governance

Domain 1: SDLC Automation (22%)

1.1 CI/CD Pipeline Implementation

AWS CodePipeline

Purpose: Fully managed continuous delivery service for automating release pipelines

Key Components:

- **Stages:** Logical divisions (Source → Build → Test → Deploy)
- **Actions:** Tasks within stages (source, build, test, deploy, approval, invoke)
- **Transitions:** Enable/disable movement between stages
- **Artifacts:** Output files passed between stages (stored in S3)

Feature	Description
Cross-account deployments	Use IAM roles to deploy to different accounts
Cross-region deployments	Deploy to multiple regions from single pipeline
Manual approvals	Add approval actions for production deployments
Parallel actions	Run multiple actions simultaneously in same stage
Pipeline variables	Pass dynamic values between stages

AWS CodeBuild

Purpose: Fully managed build service that compiles source code, runs tests, produces artifacts

Key Concepts:

- **buildspec.yml:** Build specification file defining build commands

yaml

```
version: 0.2
phases:
  install:
    runtime-versions:
      nodejs: 18
    commands:
      - npm install
  pre_build:
    commands:
      - npm run lint
  build:
    commands:
      - npm run build
      - npm test
  post_build:
    commands:
      - aws s3 cp dist/ s3://bucket/ --recursive
artifacts:
  files:
    - '**/*'
  base-directory: dist
cache:
  paths:
    - node_modules/**/*
reports:
  coverage-report:
    files:
      - '**/*'
  base-directory: coverage
```

Build Environment	Use Case
Managed images	Standard builds (Amazon Linux, Ubuntu, Windows)
Custom images	Specific dependencies, tools
Lambda compute	Fast startup, smaller builds
EC2 compute	GPU workloads, Windows containers

Best Practices:

- Use build caching to speed up builds
- Store sensitive data in Secrets Manager or Parameter Store
- Use VPC configurations for private resource access
- Enable CloudWatch Logs for debugging
- Use batch builds for parallel testing

AWS CodeDeploy

Purpose: Automate deployments to EC2, Lambda, ECS, on-premises

Deployment Types:

Platform	Deployment Types	Configuration File
EC2/On-Premises	In-place, Blue/Green	appspec.yml
Lambda	Canary, Linear, All-at-once	appspec.yml
ECS	Blue/Green	appspec.yml

appspec.yml Structure (EC2)

```
yaml
version: 0.0
os: linux
files:
  - source: /
    destination: /var/www/html
hooks:
  BeforeInstall:
    - location: scripts/install_dependencies.sh
      timeout: 300
  AfterInstall:
    - location: scripts/configure.sh
  ApplicationStart:
    - location: scripts/start_server.sh
  ValidateService:
    - location: scripts/health_check.sh
```

Deployment Lifecycle Hooks (EC2)

Phase	Hooks
Stop	ApplicationStop, DownloadBundle
Install	BeforeInstall, Install, AfterInstall
Start	ApplicationStart, ValidateService
Blue/Green Only	BeforeBlockTraffic, AfterBlockTraffic, BeforeAllowTraffic, AfterAllowTraffic

AWS CodeArtifact

Purpose: Managed artifact repository for software packages

Supported Formats: npm, PyPI, Maven, NuGet, Swift, generic

Key Features:

- Upstream repositories (connect to public registries)
- Cross-account sharing via resource policies
- Integration with CodeBuild and CodePipeline
- Automatic vulnerability scanning

AWS CodeCommit

Note: No longer accepting new customers as of July 2024. Use GitHub, GitLab, or Bitbucket as alternatives with CodePipeline.

AWS CodeCatalyst

Note: Not accepting new customers after November 7, 2025. AWS will continue maintenance but no new features. Consider using CodeBuild, CodePipeline, CodeDeploy, and CodeArtifact as alternatives.

1.2 Automated Testing in CI/CD

Testing Types by Pipeline Stage

Stage	Test Type	Purpose	Tools
Commit	Unit Tests	Test individual functions	Jest, pytest, JUnit
Build	Static Analysis	Code quality, security	SonarQube, Checkmarx
Build	Integration Tests	Test component interactions	Postman, Newman
Staging	Acceptance Tests	Validate business requirements	Selenium, Cypress
Staging	Performance Tests	Load/stress testing	Artillery, JMeter
Production	Synthetic Monitoring	Continuous validation	CloudWatch Synthetics

CodeBuild Test Reports

Supported formats: JUnit XML, Cucumber JSON, Visual Studio TRX, NUnit XML

Metrics: Pass/fail rates, duration, coverage reports

Integration: View in CodeBuild console, export to S3

CloudWatch Synthetics

Purpose: Create canaries that monitor endpoints 24/7

Use Cases:

- API endpoint monitoring
- UI workflow testing
- SSL certificate monitoring
- Broken link detection

```
javascript
```

```
const synthetics = require('Synthetics');
const log = require('SyntheticsLogger');

const apiCanary = async function () {
  const response = await synthetics.executeHttpStep(
    'Verify API',
    'https://api.example.com/health',
    { method: 'GET' }
  );
  if (response.statusCode !== 200) {
    throw new Error('API health check failed');
  }
};

exports.handler = async () => {
  return await apiCanary();
};
```

1.3 Artifact Management

Amazon ECR (Elastic Container Registry)

Purpose: Managed Docker container registry

Features:

- Image scanning (on-push or on-demand)
- Image lifecycle policies (cleanup old images)
- Cross-region replication
- Cross-account access via resource policies
- Immutable image tags

Image Scanning	Description
Basic scanning	Uses Clair (CVE database)
Enhanced scanning	Uses Amazon Inspector (deeper analysis)

EC2 Image Builder

Purpose: Automate creation of EC2 AMIs and container images

Components:

- **Image Recipe:** Base image + build components
- **Build Components:** Software installation scripts
- **Distribution Settings:** Multi-region, cross-account sharing
- **Image Pipeline:** Automated build schedule

1.4 Deployment Strategies

Strategy Comparison

Strategy	Description	Downtime	Rollback	Risk
All-at-once	Deploy to all instances simultaneously	Yes	Redeploy	High
Rolling	Deploy in batches	No	Slow	Medium
Rolling with additional batch	Add instances before rolling	No	Medium	Low
Immutable	Deploy to new instances, swap	No	Fast	Low
Blue/Green	Deploy to separate environment, switch traffic	No	Instant	Low
Canary	Deploy to small subset, gradually increase	No	Fast	Low
Traffic Splitting	Send percentage to new version	No	Fast	Low

Blue/Green Deployment Methods

Service	Method
Route 53	Weighted routing policy
ELB	Switch target groups
CloudFront	Switch origin
API Gateway	Canary deployments
CodeDeploy	Automated blue/green

Lambda Deployment Strategies (via CodeDeploy)

Canary Deployments:

Canary10Percent5Minutes - 10% shifted, wait 5 min, then 100%
Canary10Percent10Minutes - 10% shifted, wait 10 min, then 100%
Canary10Percent15Minutes - 10% shifted, wait 15 min, then 100%
Canary10Percent30Minutes - 10% shifted, wait 30 min, then 100%

Linear Deployments:

Linear10PercentEvery1Minute - 10% every minute
Linear10PercentEvery2Minutes - 10% every 2 minutes
Linear10PercentEvery3Minutes - 10% every 3 minutes
Linear10PercentEvery10Minutes - 10% every 10 minutes

All-at-once:

AllAtOnce - Shift 100% immediately

1.5 Workflow Orchestration

AWS Step Functions

Purpose: Serverless orchestration service for coordinating distributed applications and microservices

Workflow Types:

Type	Use Case	Execution Time	Billing
Standard	Long-running workflows	Up to 1 year	Per state transition
Express	High-volume, short-duration	Up to 5 minutes	Per execution + duration

Key Concepts:

- **State Machine:** Workflow definition using Amazon States Language (JSON)
- **States:** Individual steps in workflow (Task, Choice, Parallel, Wait, etc.)
- **Transitions:** Movement between states based on conditions

State Types:

State Type	Purpose	Example Use Case
Task	Execute work via Lambda, ECS, Batch, etc.	Process data, invoke API
Choice	Conditional branching	Route based on input
Parallel	Execute branches concurrently	Process multiple items simultaneously
Wait	Delay execution	Rate limiting, scheduled tasks
Map	Iterate over array	Batch processing
Pass	Transform input	Data transformation
Succeed/ Fail	Terminal states	End workflow

Example State Machine:

json

```
{
  "Comment": "Data Processing Pipeline",
  "StartAt": "ValidateInput",
  "States": {
    "ValidateInput": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:123456789012:function:ValidateData",
      "Next": "ProcessData"
    },
    "ProcessData": {
      "Type": "Parallel",
      "Branches": [
        {
          "StartAt": "TransformData",
          "States": {
            "TransformData": {
              "Type": "Task",
              "Resource": "arn:aws:lambda:us-east-1:123456789012:function:Transform",
              "End": true
            }
          }
        },
        {
          "StartAt": "EnrichData",
          "States": {
            "EnrichData": {
              "Type": "Task",
              "Resource": "arn:aws:lambda:us-east-1:123456789012:function:Enrich",
              "End": true
            }
          }
        }
      ],
      "Next": "SaveResults"
    },
    "SaveResults": {
      "Type": "Task",
      "Resource": "arn:aws:states:::dynamodb:putItem",
      "Parameters": {
        "TableName": "ProcessedData",
        "Item": {
          "id.$": "$.id",
          "data.$": "$.data"
        }
      }
    }
  }
}
```

```
},  
  "End": true  
}  
}  
}
```

Integration Capabilities:

- **AWS Lambda** - Execute functions
- **AWS Batch** - Run batch compute jobs
- **Amazon ECS/Fargate** - Run containers
- **Amazon SNS/SQS** - Messaging
- **DynamoDB** - Database operations
- **Amazon SageMaker** - Machine learning workflows
- **AWS Glue** - ETL jobs
- **Amazon Athena** - Query data

Error Handling:

```
json  
  
{  
  "Type": "Task",  
  "Resource": "arn:aws:lambda:...",  
  "Retry": [  
    {  
      "ErrorEquals": ["States.TaskFailed"],  
      "IntervalSeconds": 2,  
      "MaxAttempts": 3,  
      "BackoffRate": 2.0  
    }  
  ],  
  "Catch": [  
    {  
      "ErrorEquals": ["States.ALL"],  
      "Next": "HandleError"  
    }  
  ]  
}
```

Use Cases:

- Orchestrate microservices
- Data processing pipelines
- ETL workflows
- Human approval workflows
- Order processing systems
- Machine learning pipelines

Best Practices:

- Use Express workflows for high-volume, short-duration tasks
 - Implement proper error handling with Retry and Catch
 - Keep state machines idempotent
 - Use service integrations instead of Lambda when possible
 - Monitor executions with CloudWatch
-

Domain 2: Configuration Management and Infrastructure as Code (17%)

2.1 Infrastructure as Code Tools

AWS CloudFormation

Purpose: Model and provision AWS resources using templates

Template Structure:

yaml

AWS::TemplateFormatVersion: '2010-09-09'

Description: Sample template

Parameters:

EnvironmentName:

Type: String

Default: dev

AllowedValues:

- dev
- staging
- prod

Mappings:

RegionMap:

us-east-1:

AMI: ami-0abcdef1234567890

us-west-2:

AMI: ami-0fedcba0987654321

Conditions:

IsProduction: **!Equals** [**!Ref** EnvironmentName, prod]

Resources:

MyVPC:

Type: AWS::EC2::VPC

Properties:

CidrBlock: 10.0.0.0/16

EnableDnsHostnames: true

Tags:

- **Key:** Name
- Value:** **!Sub** \${EnvironmentName}-VPC

MyInstance:

Type: AWS::EC2::Instance

Properties:

ImageId: **!FindInMap** [RegionMap, **!Ref** 'AWS::Region', AMI]

InstanceType: **!If** [IsProduction, t3.medium, t3.micro]

SubnetId: **!Ref** MySubnet

Outputs:

VPCId:

Description: VPC ID

Value: **!Ref** MyVPC

Export:

Name: **!Sub** \${EnvironmentName}-VPC-ID

Name: !Sub \${EnvironmentName}-VPC-ID

Key Features:

Feature	Description
Change Sets	Preview changes before execution
Stack Sets	Deploy across multiple accounts/regions
Drift Detection	Identify configuration drift
Nested Stacks	Modularize templates
Cross-Stack References	Share resources via Exports/Imports
Rollback	Automatic rollback on failure

Intrinsic Functions:

Function	Purpose	Example
<code>!Ref</code>	Reference parameters/resources	<code>!Ref MyVPC</code>
<code>!GetAtt</code>	Get resource attributes	<code>!GetAtt MyInstance.PrivateIp</code>
<code>!Sub</code>	String substitution	<code>!Sub \${AWS::StackName}-resource</code>
<code>!Join</code>	Join strings	<code>!Join ['-', [dev, app]]</code>
<code>!Select</code>	Select from list	<code>!Select [0, !GetAZs ""]</code>
<code>!If</code>	Conditional logic	<code>!If [IsProduction, t3.large, t3.micro]</code>
<code>!FindInMap</code>	Retrieve mapping value	<code>!FindInMap [RegionMap, !Ref 'AWS::Region', AMI]</code>
<code>!ImportValue</code>	Import exported value	<code>!ImportValue shared-vpc-id</code>

CloudFormation StackSets:

- Deploy stacks across multiple accounts in AWS Organizations
- Use for organization-wide standards and governance
- Supports sequential and parallel deployment
- Can target specific OUs or accounts

Drift Detection:

- Identify resources modified outside CloudFormation
- Detect in individual stacks or across all stacks
- Integration with AWS Config for continuous monitoring

AWS Cloud Development Kit (CDK)

Purpose: Define infrastructure using programming languages

Supported Languages: TypeScript, JavaScript, Python, Java, C#, Go

Example (TypeScript):

typescript

```
import * as cdk from 'aws-cdk-lib';
import * as ec2 from 'aws-cdk-lib/aws-ec2';
import * as ecs from 'aws-cdk-lib/aws-ecs';
import * as ecsPatterns from 'aws-cdk-lib/aws-ecs-patterns';

export class MyStack extends cdk.Stack {
  constructor(scope: cdk.App, id: string, props?: cdk.StackProps) {
    super(scope, id, props);

    // Create VPC
    const vpc = new ec2.Vpc(this, 'MyVPC', {
      maxAzs: 2,
      natGateways: 1
    });

    // Create ECS Cluster
    const cluster = new ecs.Cluster(this, 'MyCluster', {
      vpc: vpc
    });

    // Create Fargate Service
    new ecsPatterns.ApplicationLoadBalancedFargateService(this, 'MyApp', {
      cluster: cluster,
      cpu: 512,
      memoryLimitMiB: 1024,
      desiredCount: 2,
      taskImageOptions: {
        image: ecs.ContainerImage.fromRegistry('nginx')
      }
    });
  }
}
```

Key Concepts:

Concept	Description
Constructs	Building blocks (L1: CloudFormation, L2: Curated, L3: Patterns)
Stacks	Deployable units (maps to CloudFormation stack)
Apps	Collection of stacks
Assets	Local files/Docker images bundled with app
Context	Configuration values (feature flags, AMI lookup)

CDK CLI Commands:

```
bash

cdk init app --language typescript # Initialize new project
cdk synth                          # Synthesize CloudFormation template
cdk bootstrap                      # Prepare AWS environment
cdk deploy                        # Deploy stack
cdk diff                          # Show changes
cdk destroy                       # Delete stack
cdk watch                         # Watch for changes and auto-deploy
```

Construct Levels:

Level	Description	Example
L1 (Cfn)	Direct CloudFormation mapping	CfnBucket
L2	AWS service constructs with defaults	Bucket
L3 (Patterns)	Complete solutions	ApplicationLoadBalancedFargateService

AWS SAM (Serverless Application Model)

Purpose: Framework for building serverless applications (extends CloudFormation)

Example template:

yaml

AWSTemplateFormatVersion: '2010-09-09'

Transform: AWS::Serverless-2016-10-31

Description: Serverless API

Globals:

Function:

Timeout: 30

Runtime: python3.11

Environment:

Variables:

TABLE_NAME: !Ref DynamoDBTable

Resources:

MyApi:

Type: AWS::Serverless::Api

Properties:

StageName: prod

Auth:

DefaultAuthorizer: MyCognitoAuthorizer

Authorizers:

MyCognitoAuthorizer:

UserPoolArn: !GetAtt MyCognitoUserPool.Arn

GetFunction:

Type: AWS::Serverless::Function

Properties:

CodeUri: src/

Handler: app.get_handler

Events:

GetApi:

Type: Api

Properties:

RestApiId: !Ref MyApi

Path: /items/{id}

Method: GET

Policies:

- DynamoDBReadPolicy:

TableName: !Ref DynamoDBTable

PostFunction:

Type: AWS::Serverless::Function

Properties:

CodeUri: src/

Handler: app.post_handler

Events:

Events:

PostApi:

Type: Api

Properties:

RestApiId: !Ref MyApi

Path: /items

Method: POST

Policies:

- DynamoDBCrudPolicy:

TableName: !Ref DynamoDBTable

DynamoDBTable:

Type: AWS::Serverless::SimpleTable

Properties:

PrimaryKey:

Name: id

Type: String

MyCognitoUserPool:

Type: AWS::Cognito::UserPool

Properties:

UserPoolName: MyUserPool

AutoVerifiedAttributes:

- email

Outputs:

ApiEndpoint:

Description: API Gateway endpoint URL

Value: !Sub 'https://\${MyApi}.execute-api.\${AWS::Region}.amazonaws.com/prod/'

SAM CLI Commands:

bash

sam init	# Initialize project
sam build	# Build application
sam local start-api	# Run API locally
sam local invoke	# Invoke function locally
sam deploy --guided	# Deploy with prompts
sam logs --tail	# Tail logs
sam validate	# Validate template

Key Features:

- Local testing with Docker
- Built-in deployment best practices
- Simplified syntax for serverless resources
- Integration with CodePipeline and CodeBuild

Terraform (Third-Party)

Purpose: Multi-cloud infrastructure as code tool

Key Concepts:

- **Providers:** AWS, Azure, GCP plugins
- **Resources:** Infrastructure components
- **Data Sources:** Query existing resources
- **Modules:** Reusable configurations
- **State:** Tracks deployed infrastructure

Example Configuration:

hcl

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  backend "s3" {
    bucket = "terraform-state-bucket"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt = true
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support = true

  tags = {
    Name      = "${var.environment}-vpc"
    Environment = var.environment
  }
}

resource "aws_subnet" "private" {
  count          = length(var.private_subnet_cidrs)
  vpc_id         = aws_vpc.main.id
  cidr_block     = var.private_subnet_cidrs[count.index]
  availability_zone = data.aws_availability_zones.available.names[count.index]

  tags = {
    Name = "${var.environment}-private-subnet-${count.index + 1}"
  }
}

module "alb" {
  source = "./modules/alb"
```

```

vpc_id      = aws_vpc.main.id
subnet_ids  = aws_subnet.public[*].id
security_groups = [aws_security_group.alb.id]

environment = var.environment
}

output "vpc_id" {
  value = aws_vpc.main.id
}

```

Terraform Commands:

```

bash

terraform init      # Initialize working directory
terraform plan      # Preview changes
terraform apply     # Apply changes
terraform destroy   # Destroy infrastructure
terraform fmt       # Format configuration
terraform validate  # Validate configuration
terraform state list # List resources in state
terraform import    # Import existing resources

```

State Management:

- Store state remotely in S3 with DynamoDB locking
- Use workspaces for environment isolation
- Enable state locking to prevent conflicts
- Backup state files regularly

2.2 Configuration Management

AWS Systems Manager

Purpose: Unified interface for managing AWS resources

Key Components:

Parameter Store

Purpose: Secure, hierarchical storage for configuration data

Parameter Types:

Type	Description	Use Case
String	Plain text	Configuration values
StringList	Comma-separated values	Lists
SecureString	KMS-encrypted	Sensitive data

Hierarchical Organization:

```
/myapp/dev/database/connection-string  
/myapp/dev/api/key  
/myapp/prod/database/connection-string  
/myapp/prod/api/key
```

Access in Applications:

```
python  
  
import boto3  
  
ssm = boto3.client('ssm')  
  
# Get single parameter  
response = ssm.get_parameter(  
    Name='/myapp/prod/database/connection-string',  
    WithDecryption=True  
)  
value = response['Parameter']['Value']  
  
# Get parameters by path  
response = ssm.get_parameters_by_path(  
    Path='/myapp/prod',  
    Recursive=True,  
    WithDecryption=True  
)
```

Features:

- Free tier available (Standard parameters)
- Advanced parameters: larger size, parameter policies
- Integration with CloudFormation
- Version history
- Parameter policies for expiration notifications

Session Manager

Purpose: Secure shell access without bastion hosts

Benefits:

- No open inbound ports
- No SSH key management
- Centralized access control via IAM
- Session logging to S3 and CloudWatch
- Integration with CloudTrail

Requirements:

- SSM Agent installed on instances
- IAM instance profile with permissions
- Outbound HTTPS access to Systems Manager endpoints

Patch Manager

Purpose: Automate OS patching across EC2 and on-premises

Patch Baselines:

- Pre-defined baselines for each OS
- Custom baselines with approval rules
- Patch groups for different environments

Maintenance Windows:

- Schedule patching operations
- Define targets and tasks
- Set maximum concurrency and error threshold

Run Command

Purpose: Remotely execute commands on managed instances

Use Cases:

- Run shell scripts
- Install applications
- Configure instances
- Gather system information

Example:

```
bash
```

```
aws ssm send-command \  
  --document-name "AWS-RunShellScript" \  
  --targets "Key=tag:Environment,Values=production" \  
  --parameters 'commands=["sudo systemctl restart nginx"]' \  
  --output text
```

Automation

Purpose: Simplify common maintenance and deployment tasks

Built-in Runbooks:

- `AWS-UpdateLinuxAmi` - Create updated AMIs
- `AWS-RestartEC2Instance` - Restart instances
- `AWS-CreateSnapshot` - Create EBS snapshots
- `AWS-AttachIAMToInstance` - Attach IAM role

Custom Automation Documents:

yaml

```
schemaVersion: '0.3'
description: Custom automation to stop and start instance
parameters:
  InstanceId:
    type: String
    description: EC2 instance ID
mainSteps:
  - name: StopInstance
    action: aws:executeAwsApi
    inputs:
      Service: ec2
      Api: StopInstances
      InstanceIds:
        - '{{ InstanceId }}'
  - name: WaitForStop
    action: aws:waitForAwsResourceProperty
    inputs:
      Service: ec2
      Api: DescribeInstances
      InstanceIds:
        - '{{ InstanceId }}'
      PropertySelector: '$.Reservations[0].Instances[0].State.Name'
      DesiredValues:
        - stopped
  - name: StartInstance
    action: aws:executeAwsApi
    inputs:
      Service: ec2
      Api: StartInstances
      InstanceIds:
        - '{{ InstanceId }}'
```

State Manager

Purpose: Maintain consistent configuration state

Use Cases:

- Ensure software installation
- Configure CloudWatch agent
- Join domain
- Run antivirus scans

Association:

- Links document to targets
- Defines execution schedule
- Specifies compliance severity

Inventory

Purpose: Collect metadata about managed instances

Collected Data:

- Applications
- AWS components
- Network configuration
- Windows updates
- Instance details
- Services
- Windows registry

Integration:

- Query with AWS Config
- Analyze with Athena
- Visualize with QuickSight

Compliance

Purpose: View compliance status across resources

Compliance Types:

- Patch compliance
- Association compliance
- Custom compliance

2.3 Container Configuration

Amazon ECS Configuration

Task Definition Example:

json

```
{
  "family": "my-app",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "256",
  "memory": "512",
  "containerDefinitions": [
    {
      "name": "app",
      "image": "123456789012.dkr.ecr.us-east-1.amazonaws.com/my-app:latest",
      "portMappings": [
        {
          "containerPort": 80,
          "protocol": "tcp"
        }
      ],
      "environment": [
        {
          "name": "ENVIRONMENT",
          "value": "production"
        }
      ],
      "secrets": [
        {
          "name": "DB_PASSWORD",
          "valueFrom": "arn:aws:secretsmanager:us-east-1:123456789012:secret:db-password"
        }
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/my-app",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "ecs"
        }
      },
      "healthCheck": {
        "command": ["CMD-SHELL", "curl -f http://localhost/health || exit 1"],
        "interval": 30,
        "timeout": 5,
        "retries": 3,
        "startPeriod": 60
      }
    }
  ]
}
```

```
]
}
```

Amazon EKS Configuration

kubectl Configuration:

```
bash

# Update kubeconfig
aws eks update-kubeconfig --name my-cluster --region us-east-1

# Deploy application
kubectl apply -f deployment.yaml
```

Example Deployment:

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      serviceAccountName: my-app-sa
      containers:
        - name: app
          image: 123456789012.dkr.ecr.us-east-1.amazonaws.com/my-app:latest
          ports:
            - containerPort: 8080
          env:
            - name: ENVIRONMENT
              value: "production"
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: password
      resources:
        requests:
          memory: "256Mi"
          cpu: "250m"
        limits:
          memory: "512Mi"
          cpu: "500m"
      livenessProbe:
        httpGet:
          path: /health
          port: 8080
        initialDelaySeconds: 30
        periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /ready
```

```
    path: /ready
    port: 8080
    initialDelaySeconds: 5
    periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: my-app
  namespace: production
spec:
  type: LoadBalancer
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080
```

EKS Managed Node Groups:

- Automated provisioning and lifecycle management
- Integration with EC2 Auto Scaling
- Support for Spot instances
- Custom AMIs supported

Fargate for EKS:

- Serverless compute for pods
- No node management required
- Pay per pod
- Isolated compute environment

Domain 3: Resilient Cloud Solutions (15%)

3.1 High Availability Architectures

Multi-AZ Deployments

Compute:

- **EC2:** Deploy across multiple AZs with Auto Scaling
- **ECS:** Distribute tasks across AZs
- **EKS:** Node groups span multiple AZs
- **Lambda:** Automatically runs across AZs

Database:

- **RDS:** Multi-AZ for synchronous replication
- **Aurora:** Multi-AZ by default, up to 15 read replicas
- **DynamoDB:** Automatically replicated across 3 AZs
- **ElastiCache:** Multi-AZ with automatic failover

Storage:

- **EBS:** Single AZ (use snapshots for backup)
- **EFS:** Multi-AZ file system
- **S3:** Automatically replicated across multiple AZs

Load Balancing

Application Load Balancer (ALB)

Features:

- Layer 7 (HTTP/HTTPS) routing
- Host-based and path-based routing
- WebSocket support
- HTTP/2 and gRPC support
- Integration with AWS WAF
- Sticky sessions
- TLS termination

Target Types:

- EC2 instances
- IP addresses
- Lambda functions
- ECS tasks

Routing Rules Example:

yaml

Listener:

- Port: 443

Rules:

- Condition: Host = api.example.com

Action: Forward to API target group

- Condition: Path = /images/*

Action: Forward to Image target group

- Condition: Query = version=v2

Action: Forward to V2 target group

- Default:

Action: Forward to Default target group

Network Load Balancer (NLB)

Features:

- Layer 4 (TCP/UDP/TLS) routing
- Ultra-low latency
- Static IP addresses
- Preserve source IP
- Millions of requests per second
- TLS termination

Use Cases:

- Extreme performance requirements
- Static IP needed
- TCP/UDP protocols
- PrivateLink services

Gateway Load Balancer (GWLB)

Purpose: Deploy, scale, and manage third-party virtual appliances

Use Cases:

- Firewalls
- Intrusion detection/prevention
- Deep packet inspection

Auto Scaling

EC2 Auto Scaling

Scaling Policies:

Type	Description	Use Case
Target Tracking	Maintain metric at target value	Keep CPU at 50%
Step Scaling	Scale based on metric thresholds	Add 2 instances when CPU > 70%
Simple Scaling	Single adjustment	Legacy, use step scaling instead
Scheduled	Scale at specific times	Known traffic patterns
Predictive	ML-based forecasting	Variable traffic patterns

Launch Templates vs Launch Configurations:

Feature	Launch Template	Launch Configuration
Versioning	Yes	No
Multiple instance types	Yes	No
Spot Fleet support	Yes	No
T2/T3 unlimited	Yes	No
Status	Recommended	Legacy

Auto Scaling Group Configuration:

json

```
{
  "AutoScalingGroupName": "my-asg",
  "LaunchTemplate": {
    "LaunchTemplateId": "lt-1234567890abcdef0",
    "Version": "$Latest"
  },
  "MinSize": 2,
  "MaxSize": 10,
  "DesiredCapacity": 4,
  "DefaultCooldown": 300,
  "HealthCheckType": "ELB",
  "HealthCheckGracePeriod": 300,
  "VPCZoneIdentifier": "subnet-1,subnet-2,subnet-3",
  "TargetGroupARNs": ["arn:aws:elasticloadbalancing:..."],
  "Tags": [
    {
      "Key": "Environment",
      "Value": "Production",
      "PropagateAtLaunch": true
    }
  ]
}
```

Lifecycle Hooks:

- Pause instance launch/termination
- Perform custom actions
- Wait for completion signal
- Timeout configurable

Instance Warm-up:

- Don't count instance metrics during warm-up period
- Prevents premature scale-in decisions
- Configurable per scaling policy

Application Auto Scaling

Supported Services:

- ECS services
- DynamoDB tables and GSIs
- Aurora replicas
- SageMaker endpoints
- Lambda concurrency
- Comprehend endpoints
- AppStream 2.0 fleets

Example (ECS Service):

```
bash

# Register scalable target
aws application-autoscaling register-scalable-target \
  --service-namespace ecs \
  --resource-id service/my-cluster/my-service \
  --scalable-dimension ecs:service:DesiredCount \
  --min-capacity 2 \
  --max-capacity 10

# Create scaling policy
aws application-autoscaling put-scaling-policy \
  --service-namespace ecs \
  --resource-id service/my-cluster/my-service \
  --scalable-dimension ecs:service:DesiredCount \
  --policy-name cpu-scaling \
  --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration file://config.json
```

config.json:

```
json

{
  "TargetValue": 75.0,
  "PredefinedMetricSpecification": {
    "PredefinedMetricType": "ECSServiceAverageCPUUtilization"
  },
  "ScaleOutCooldown": 60,
  "ScaleInCooldown": 300
}
```

3.2 Disaster Recovery Strategies

DR Strategy Comparison

Strategy	RTO	RPO	Cost	Complexity	AWS Services
Backup & Restore	Hours	Hours	Low	Low	S3, Glacier, Backup
Pilot Light	Minutes	Minutes	Medium	Medium	Route 53, RDS standby
Warm Standby	Minutes	Seconds	High	Medium	ASG (min capacity), RDS Multi-AZ
Multi-Site Active-Active	Seconds	None	Very High	High	Route 53, Global Accelerator

Backup & Restore

Strategy: Regular backups to S3/Glacier, restore when needed

Implementation:

- Database snapshots
- EBS snapshots
- S3 versioning and replication
- AWS Backup for centralized management

Example:

```
bash
```

```
# Automated backup with AWS Backup
```

```
aws backup create-backup-plan \  
--backup-plan file://backup-plan.json
```

```
# Backup plan for daily snapshots
```

```
{  
  "BackupPlanName": "DailyBackup",  
  "Rules": [  
    {  
      "RuleName": "DailyBackupRule",  
      "TargetBackupVaultName": "Default",  
      "ScheduleExpression": "cron(0 5 ? * * *)",  
      "StartWindowMinutes": 60,  
      "CompletionWindowMinutes": 120,  
      "Lifecycle": {  
        "DeleteAfterDays": 30,  
        "MoveToColdStorageAfterDays": 7  
      }  
    }  
  ]  
}
```

Pilot Light

Strategy: Minimal version always running, scale up when needed

Core Components:

- Database replication to DR region
- Critical data continuously synchronized
- AMIs and templates ready
- Route 53 for DNS failover

Implementation:

1. Keep database running with read replicas
2. Store AMIs and infrastructure templates
3. Automate scale-up scripts
4. Monitor primary with health checks
5. Failover via Route 53 on failure

Warm Standby

Strategy: Scaled-down but fully functional version running

Characteristics:

- All services running at minimum capacity
- Fast recovery time
- Continuous synchronization
- Higher cost than pilot light

Implementation:

- ASG with minimum capacity
- RDS Multi-AZ or cross-region read replica with promotion
- Load balancers configured
- Route 53 weighted routing (0% to standby)

Multi-Site Active-Active

Strategy: Full-scale deployment in multiple regions

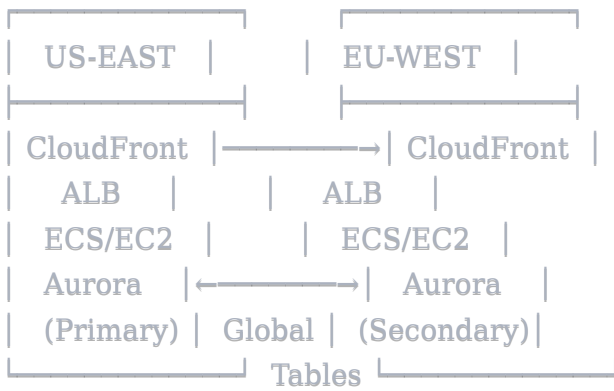
Benefits:

- Lowest RTO/RPO
- Geographic load distribution
- Best user experience
- No failover required

Implementation:

- Global Accelerator for intelligent routing
- Route 53 geolocation/latency routing
- DynamoDB Global Tables
- Aurora Global Database
- S3 Cross-Region Replication
- CloudFront for static content

Example Architecture:



Cross-Region Replication

DynamoDB Global Tables:

- Multi-region, multi-active replication
- Read/write to any region
- Automatic conflict resolution
- Sub-second replication

Aurora Global Database:

- Up to 5 secondary regions
- Sub-second replication latency
- Read replicas in secondary regions
- Fast failover (< 1 minute)

S3 Cross-Region Replication (CRR):

- Automatic, asynchronous replication
- Entire bucket or prefix-based
- Replica modification sync
- Bidirectional replication

RDS Cross-Region Read Replicas:

- Asynchronous replication
- Promote to standalone database
- DR and read scaling

3.3 Resilience Patterns

Circuit Breaker Pattern

Purpose: Prevent cascading failures by failing fast

Implementation with Step Functions:

```
json
{
  "Type": "Task",
  "Resource": "arn:aws:lambda:...",
  "Retry": [
    {
      "ErrorEquals": ["States.TaskFailed"],
      "IntervalSeconds": 2,
      "MaxAttempts": 3,
      "BackoffRate": 2.0
    }
  ],
  "Catch": [
    {
      "ErrorEquals": ["States.ALL"],
      "ResultPath": "$.error",
      "Next": "CircuitBreakerOpen"
    }
  ]
}
```

Retry with Exponential Backoff

Purpose: Handle transient failures gracefully

AWS SDK Example (Python):

```
python

import boto3
from botocore.config import Config

config = Config(
    retries={
        'max_attempts': 10,
        'mode': 'adaptive'
    }
)

client = boto3.client('dynamodb', config=config)
```

Bulkhead Pattern

Purpose: Isolate resources to prevent total system failure

Implementation:

- Separate Lambda functions per service
- Different DynamoDB tables per domain
- VPC subnets per application tier
- Separate Auto Scaling groups

Throttling and Rate Limiting

API Gateway:

- Per-method throttle limits
- Burst limits
- Usage plans and API keys

Lambda:

- Reserved concurrency per function
- Account-level concurrency limits

DynamoDB:

- Provisioned capacity units
- On-demand scaling
- Burst capacity

Health Checks

Types:

Type	Purpose	Check Interval
ELB Health Check	Application availability	5-300 seconds
Route 53 Health Check	Endpoint reachability	10 or 30 seconds
Auto Scaling Health Check	Instance health	Configurable
Container Health Check	Container health	Configurable

Multi-Layer Health Checks:



Domain 4: Monitoring and Logging (15%)

4.1 CloudWatch Monitoring

CloudWatch Metrics

Default Metrics by Service:

Service	Key Metrics	Frequency
EC2	CPU, Network, Disk	5 min (1 min with detailed)
EBS	Read/Write ops, throughput	5 min
ELB	Requests, latency, errors	1 min
RDS	CPU, connections, IOPS	1 min
Lambda	Invocations, duration, errors	1 min
DynamoDB	Consumed capacity, throttles	1 min

Custom Metrics:

```
bash

# Put metric data
aws cloudwatch put-metric-data \
  --namespace "MyApp" \
  --metric-name "PageLoadTime" \
  --value 245 \
  --unit Milliseconds \
  --dimensions Environment=Production,Page=Home

# Put metric with timestamp
aws cloudwatch put-metric-data \
  --namespace "MyApp" \
  --metric-name "CacheHitRate" \
  --value 95.5 \
  --unit Percent \
  --timestamp 2024-12-04T10:00:00Z
```

Python Example:

```
python
```

```
import boto3
from datetime import datetime

cloudwatch = boto3.client('cloudwatch')

cloudwatch.put_metric_data(
    Namespace='MyApp',
    MetricData=[
        {
            'MetricName': 'ProcessingTime',
            'Value': 123.45,
            'Unit': 'Milliseconds',
            'Timestamp': datetime.utcnow(),
            'Dimensions': [
                {'Name': 'Environment', 'Value': 'Production'},
                {'Name': 'Service', 'Value': 'API'}
            ]
        }
    ]
)
```

High-Resolution Metrics:

- 1-second granularity
- Store for 3 hours at 1-second resolution
- Useful for real-time monitoring
- Higher cost than standard metrics

CloudWatch Alarms

Alarm States:

- **OK:** Metric within threshold
- **ALARM:** Metric breached threshold
- **INSUFFICIENT_DATA:** Not enough data

Alarm Configuration:

json

```
{
  "AlarmName": "HighCPUUtilization",
  "ComparisonOperator": "GreaterThanThreshold",
  "EvaluationPeriods": 2,
  "MetricName": "CPUUtilization",
  "Namespace": "AWS/EC2",
  "Period": 300,
  "Statistic": "Average",
  "Threshold": 80.0,
  "ActionsEnabled": true,
  "AlarmActions": [
    "arn:aws:sns:us-east-1:123456789012:critical-alerts",
    "arn:aws:autoscaling:us-east-1:123456789012:scalingPolicy:..."
  ],
  "AlarmDescription": "Alert when CPU exceeds 80% for 10 minutes",
  "Dimensions": [
    {
      "Name": "InstanceId",
      "Value": "i-1234567890abcdef0"
    }
  ]
}
```

Composite Alarms:

- Combine multiple alarms using AND/OR logic
- Reduce alarm fatigue
- Complex conditions

Example:

bash

```
aws cloudwatch put-composite-alarm \
  --alarm-name "HighTrafficAndErrors" \
  --alarm-rule "ALARM(HighRequestCount) AND ALARM(HighErrorRate)" \
  --actions-enabled \
  --alarm-actions "arn:aws:sns:us-east-1:123456789012:critical"
```

Alarm Actions:

- SNS notifications
- Auto Scaling actions
- EC2 actions (stop, terminate, reboot, recover)
- Systems Manager OpsItem creation

Anomaly Detection:

- ML-based anomaly detection
- Automatically creates dynamic thresholds
- Adapts to trends and patterns

```
bash
```

```
aws cloudwatch put-anomaly-detector \  
  --namespace AWS/EC2 \  
  --metric-name CPUUtilization \  
  --dimensions Name=InstanceId,Value=i-1234567890abcdef0 \  
  --stat Average
```

CloudWatch Dashboards

Features:

- Custom visualization
- Multiple metrics on single graph
- Cross-region and cross-account
- Auto-refresh
- Public sharing

Dashboard Example:

json

```
{
  "widgets": [
    {
      "type": "metric",
      "properties": {
        "metrics": [
          ["AWS/EC2", "CPUUtilization", {"stat": "Average"}],
          [".", "NetworkIn", {"stat": "Sum"}]
        ],
        "period": 300,
        "stat": "Average",
        "region": "us-east-1",
        "title": "EC2 Instance Metrics",
        "yAxis": {
          "left": {"min": 0, "max": 100}
        }
      }
    },
    {
      "type": "log",
      "properties": {
        "query": "SOURCE '/aws/lambda/my-function' | fields @timestamp, @message | sort @timestamp",
        "region": "us-east-1",
        "title": "Recent Lambda Logs"
      }
    }
  ]
}
```

4.2 CloudWatch Logs

Log Groups and Streams

Hierarchy:

Log Group: /aws/lambda/my-function

└─ Log Stream: 2024/12/04/[\$LATEST]abc123

└─ Log Stream: 2024/12/04/[\$LATEST]def456

└─ Log Stream: 2024/12/04/[\$LATEST]ghi789

Log Retention:

- Configurable: 1 day to 10 years
- Never expire option
- Set per log group

Log Groups by Service:

Service	Log Group Pattern
Lambda	/aws/lambda/function-name
ECS	/ecs/cluster-name
API Gateway	API-Gateway-Execution-Logs_api-id/stage
RDS	/aws/rds/instance/instance-name/error
CodeBuild	/aws/codebuild/project-name

CloudWatch Logs Insights

Query Language Example:

```
# Find errors in Lambda logs
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 20

# Analyze API latency
fields @timestamp, requestId, duration
| filter ispresent(duration)
| stats avg(duration), max(duration), min(duration) by bin(5m)

# Count errors by error type
fields @timestamp, @message
| parse @message "Error: *" as errorType
| stats count() by errorType

# Find slow requests
fields @timestamp, @requestId, @duration
| filter @duration > 1000
| sort @duration desc
| limit 50

# Calculate percentiles
stats avg(@duration),
    percentile(@duration, 50),
    percentile(@duration, 95),
    percentile(@duration, 99)
```

Saved Queries:

- Reuse common queries
- Share across team
- Version control with CloudFormation

Log Subscriptions

Purpose: Stream logs to other services for processing

Destinations:

Destination	Use Case
Lambda	Real-time processing
Kinesis Data Streams	Real-time analytics
Kinesis Data Firehose	S3/Elasticsearch delivery
Another Account	Centralized logging

Example (Lambda Subscription):

```
bash

aws logs put-subscription-filter \
  --log-group-name "/aws/lambda/my-function" \
  --filter-name "ErrorFilter" \
  --filter-pattern "ERROR" \
  --destination-arn "arn:aws:lambda:us-east-1:123456789012:function:log-processor"
```

Cross-Account Log Aggregation:

1. Create destination in central account
2. Add resource policy to destination
3. Create subscription filter in source accounts
4. Stream logs to central location

Metric Filters

Purpose: Extract metrics from log data

Example:

```
bash
```

```
# Create metric filter for 404 errors
```

```
aws logs put-metric-filter \  
  --log-group-name "/aws/elasticbeanstalk/my-app/var/log/nginx/access.log" \  
  --filter-name "404Errors" \  
  --filter-pattern '[ip, id, user, timestamp, request, status_code=404, size]' \  
  --metric-transformations \  
    metricName=404Count,metricNamespace=MyApp,metricValue=1,defaultValue=0
```

Filter Patterns:

Pattern Type	Example
Simple match	"ERROR"
Multiple terms	"ERROR" "WARN"
Space-delimited	[ip, timestamp, request]
JSON	{ \$.statusCode = 404 }
Numeric	[..., status_code=5*, ...]

CloudWatch Agent

Purpose: Collect system-level metrics and logs from EC2 and on-premises

Collected Metrics:

- Memory utilization
- Disk space utilization
- Disk I/O
- Network metrics
- Process metrics
- System uptime

Configuration:

json

```
{
  "metrics": {
    "namespace": "MyCustomNamespace",
    "metrics_collected": {
      "cpu": {
        "measurement": [
          {"name": "cpu_usage_idle", "rename": "CPU_IDLE", "unit": "Percent"},
          "cpu_usage_iowait"
        ],
        "totalcpu": false
      },
      "disk": {
        "measurement": [
          {"name": "used_percent", "rename": "DISK_USED", "unit": "Percent"}
        ],
        "resources": ["/", "/mnt"]
      },
      "mem": {
        "measurement": [
          {"name": "mem_used_percent", "rename": "MEM_USED", "unit": "Percent"}
        ]
      }
    }
  },
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/nginx/access.log",
            "log_group_name": "/ec2/nginx/access",
            "log_stream_name": "{instance_id}"
          }
        ]
      }
    }
  }
}
```

Deployment Methods:

- Systems Manager Run Command
 - CloudFormation
 - User data script
 - Configuration management tools
-

4.3 AWS X-Ray

Purpose and Concepts

Purpose: Distributed tracing for microservices applications

Key Concepts:

- **Trace:** End-to-end journey of request
- **Segment:** Work done by single service
- **Subsegment:** Work done within segment (DB call, HTTP request)
- **Annotation:** Indexed key-value pairs for filtering
- **Metadata:** Non-indexed key-value pairs for additional context

X-Ray Architecture

Application → X-Ray SDK → X-Ray Daemon → X-Ray Service

Components:

1. **X-Ray SDK:** Instrument application code
2. **X-Ray Daemon:** Receives segments and sends to X-Ray service
3. **X-Ray Service:** Processes and stores traces

Instrumentation

Supported Languages: Java, .NET, Node.js, Python, Ruby, Go, PHP

Python Example:

python

```
from aws_xray_sdk.core import xray_recorder
from aws_xray_sdk.core import patch_all
import boto3

# Patch all supported libraries
patch_all()

# Custom segment
@xray_recorder.capture('process_payment')
def process_payment(amount):
    # Add annotation (indexed)
    xray_recorder.put_annotation('payment_amount', amount)

    # Add metadata (not indexed)
    xray_recorder.put_metadata('customer', {'id': '12345', 'tier': 'premium'})

    # Your business logic
    result = charge_card(amount)
    return result

# Subsegment for HTTP call
with xray_recorder.capture('external_api_call'):
    response = requests.get('https://api.example.com/data')
```

Node.js Example:

javascript

```
const AWSXRay = require('aws-xray-sdk-core');
const AWS = AWSXRay.captureAWS(require('aws-sdk'));
const http = AWSXRay.captureHTTPs(require('http'));

// Express middleware
const app = require('express')();
app.use(AWSXRay.express.openSegment('MyApp'));

app.get('/api/users/:id', async (req, res) => {
  const segment = AWSXRay.getSegment();
  const subsegment = segment.addNewSubsegment('fetch-user');

  try {
    subsegment.addAnnotation('userId', req.params.id);
    const user = await fetchUser(req.params.id);
    res.json(user);
  } finally {
    subsegment.close();
  }
});

app.use(AWSXRay.express.closeSegment());
```

Service Map

Visualization:

- Service dependencies
- Average latency
- Request rate
- Error rate
- Throttling

Use Cases:

- Identify bottlenecks
- Understand service dependencies
- Detect anomalies
- Plan optimizations

Trace Analysis

Filter Expressions:

```
# Find traces with errors
fault = true

# Find slow requests
responseTime > 5

# Filter by annotation
annotation.payment_amount > 1000

# Combine conditions
http.status = 500 AND service("payment-service")

# User-specific traces
user = "user@example.com"
```

Group Traces:

- Create groups based on filter expressions
- Monitor specific subsets
- Set alarms on groups

X-Ray Integration

AWS Services with Native Support:

- API Gateway
- Lambda
- Elastic Beanstalk
- ECS
- EC2 (with X-Ray daemon)
- App Runner

Configuration Examples:

Lambda:

```
yaml

MyFunction:
  Type: AWS::Serverless::Function
  Properties:
    Runtime: python3.11
    Handler: app.handler
    Tracing: Active # Enable X-Ray
```

API Gateway:

```
bash
```

```
aws apigateway update-stage \  
  --rest-api-id abc123 \  
  --stage-name prod \  
  --patch-operations op=replace,path=/tracingEnabled,value=true
```

ECS Task Definition:

```
json
```

```
{  
  "containerDefinitions": [  
    {  
      "name": "xray-daemon",  
      "image": "amazon/aws-xray-daemon",  
      "cpu": 32,  
      "memoryReservation": 256,  
      "portMappings": [  
        {  
          "containerPort": 2000,  
          "protocol": "udp"  
        }  
      ]  
    },  
    {  
      "name": "app",  
      "image": "my-app:latest",  
      "environment": [  
        {  
          "name": "AWS_XRAY_DAEMON_ADDRESS",  
          "value": "xray-daemon:2000"  
        }  
      ]  
    }  
  ]  
}
```

Sampling Rules

Purpose: Control cost by sampling traces

Default Rule: 1 request per second, plus 5% of additional requests

Custom Rules:

json

```
{
  "version": 2,
  "rules": [
    {
      "description": "High-value transactions",
      "host": "*",
      "http_method": "*",
      "url_path": "/api/payment/*",
      "fixed_target": 10,
      "rate": 1.0,
      "priority": 100
    },
    {
      "description": "Default sampling",
      "host": "*",
      "http_method": "*",
      "url_path": "*",
      "fixed_target": 1,
      "rate": 0.05,
      "priority": 1000
    }
  ],
  "default": {
    "fixed_target": 1,
    "rate": 0.05
  }
}
```

4.4 Amazon CloudWatch Application Insights

Purpose: Automated monitoring and troubleshooting for applications

Supported Applications:

- SQL Server on EC2 or RDS
- .NET applications on IIS
- Java applications
- Custom applications

Features:

- Automated dashboards
- Anomaly detection
- Problem detection
- Automated analysis
- Root cause identification

Configuration:

```
bash

# Create application
aws application-insights create-application \
  --resource-group-name "MyApp" \
  --ops-center-enabled

# Configure component monitoring
aws application-insights update-component-configuration \
  --resource-group-name "MyApp" \
  --component-name "MyWebServer" \
  --monitor \
  --tier "DOT_NET_WEB_TIER"
```

Domain 5: Incident and Event Response (14%)

5.1 Event-Driven Automation

Amazon EventBridge

Purpose: Serverless event bus for application integration

Event Sources:

- AWS services (90+ supported)
- Custom applications
- SaaS applications (Salesforce, Datadog, etc.)
- Schedule-based (cron)

Architecture:

```
Event Source → Event Bus → Rules → Targets
```

Event Pattern Examples:

EC2 State Change:

```
json
{
  "source": ["aws.ec2"],
  "detail-type": ["EC2 Instance State-change Notification"],
  "detail": {
    "state": ["terminated"]
  }
}
```

S3 Object Created:

```
json
{
  "source": ["aws.s3"],
  "detail-type": ["Object Created"],
  "detail": {
    "bucket": {
      "name": ["my-bucket"]
    },
    "object": {
      "key": [{
        "prefix": "uploads/"
      }]
    }
  }
}
```

CloudWatch Alarm State Change:

```
json
{
  "source": ["aws.cloudwatch"],
  "detail-type": ["CloudWatch Alarm State Change"],
  "detail": {
    "alarmName": ["HighCPU"],
    "state": {
      "value": ["ALARM"]
    }
  }
}
```

Scheduled Events (Cron):

```
bash
```

```
# Every day at 9 AM UTC
```

```
rate(1 day)
```

```
cron(0 9 * * ? *)
```

```
# Every 5 minutes
```

```
rate(5 minutes)
```

```
# Monday through Friday at 6 PM
```

```
cron(0 18 ? * MON-FRI *)
```

```
# First day of month at midnight
```

```
cron(0 0 1 * ? *)
```

Target Types:

- Lambda functions
- Step Functions state machines
- SNS topics
- SQS queues
- ECS tasks
- CodePipeline
- CodeBuild projects
- Systems Manager Run Command
- Systems Manager Automation
- API Gateway
- Kinesis streams
- EventBridge API destinations (HTTP endpoints)

Rule Configuration:

json

```
{
  "Name": "AutomateInstanceRecovery",
  "EventPattern": {
    "source": ["aws.ec2"],
    "detail-type": ["EC2 Instance State-change Notification"],
    "detail": {
      "state": ["stopped"]
    }
  },
  "Targets": [
    {
      "Arn": "arn:aws:lambda:us-east-1:123456789012:function:RecoverInstance",
      "Id": "1"
    },
    {
      "Arn": "arn:aws:sns:us-east-1:123456789012:admin-alerts",
      "Id": "2",
      "InputTransformer": {
        "InputPathsMap": {
          "instance": "$.detail.instance-id",
          "state": "$.detail.state"
        },
        "InputTemplate": "\"Instance <instance> has entered <state> state\""
      }
    }
  ]
}
```

Custom Events:

```
python
```

```
import boto3
```

```
import json
```

```
eventbridge = boto3.client('events')
```

```
response = eventbridge.put_events(  
    Entries=[  
        {  
            'Source': 'myapp.orders',  
            'DetailType': 'Order Placed',  
            'Detail': json.dumps({  
                'orderId': '12345',  
                'amount': 99.99,  
                'customerId': 'user@example.com'  
            }),  
            'EventBusName': 'default'  
        }  
    ]  
)
```

Cross-Account Event Delivery:

1. Create event bus in target account
2. Add resource policy to event bus
3. Create rule in source account with target event bus

Event Replay:

- Archive events automatically
- Replay events for debugging or recovery
- Configurable retention (indefinite or days)

CloudWatch Events vs EventBridge

Feature	CloudWatch Events	EventBridge
Event buses	Default only	Multiple custom buses
Schema registry	No	Yes
SaaS integration	No	Yes
Cross-account	Limited	Enhanced
API destinations	No	Yes
Status	Legacy	Preferred

5.2 Automated Remediation

AWS Config Remediation

Purpose: Automatically fix non-compliant resources

Built-in Remediation Actions:

- Enable S3 bucket encryption
- Enable CloudTrail logging
- Remove unused security groups
- Enable VPC Flow Logs
- Attach IAM policies

Custom Remediation (SSM Automation):

Config Rule:

```
json
{
  "ConfigRuleName": "required-tags",
  "Source": {
    "Owner": "AWS",
    "SourceIdentifier": "REQUIRED_TAGS"
  },
  "InputParameters": {
    "tag1Key": "Environment",
    "tag2Key": "Owner"
  }
}
```

Remediation Configuration:

json

```
{
  "ConfigRuleName": "required-tags",
  "TargetType": "SSM_DOCUMENT",
  "TargetIdentifier": "AWS-CreateTags",
  "Parameters": {
    "ResourceARN": {
      "ResourceValue": {
        "Value": "RESOURCE_ID"
      }
    },
    "Tags": {
      "StaticValue": {
        "Values": [
          "{\"Key\":\"Environment\",\"Value\":\"Production\"}",
          "{\"Key\":\"Owner\",\"Value\":\"DevOps\"}"
        ]
      }
    }
  },
  "Automatic": true,
  "MaximumAutomaticAttempts": 5,
  "RetryAttemptSeconds": 60
}
```

Custom SSM Automation Document:

yaml

```
schemaVersion: '0.3'
description: Remediate unencrypted S3 bucket
parameters:
  BucketName:
    type: String
    description: S3 bucket to encrypt
mainSteps:
  - name: EnableEncryption
    action: aws:executeAwsApi
    inputs:
      Service: s3
      Api: PutBucketEncryption
      Bucket: '{{ BucketName }}'
      ServerSideEncryptionConfiguration:
        Rules:
          - ApplyServerSideEncryptionByDefault:
              SSEAlgorithm: AES256
  - name: VerifyEncryption
    action: aws:assertAwsResourceProperty
    inputs:
      Service: s3
      Api: GetBucketEncryption
      Bucket: '{{ BucketName }}'
      PropertySelector: '$.ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionB
      DesiredValues:
        - AES256
```

EventBridge-Triggered Remediation

Pattern:

CloudWatch Alarm → EventBridge → Lambda/Step Functions → Remediation

Example: Auto-restart unhealthy ECS tasks

python

```
import boto3
import json

ecs = boto3.client('ecs')
sns = boto3.client('sns')

def lambda_handler(event, context):
    # Parse alarm details
    alarm_name = event['detail']['alarmName']

    # Get task details from alarm dimensions
    cluster = event['detail']['configuration']['metrics'][0]['metricStat']['metric']['dimensions']['ClusterName']
    service = event['detail']['configuration']['metrics'][0]['metricStat']['metric']['dimensions']['ServiceName']

    # Force new deployment
    response = ecs.update_service(
        cluster=cluster,
        service=service,
        forceNewDeployment=True
    )

    # Send notification
    sns.publish(
        TopicArn='arn:aws:sns:us-east-1:123456789012:ops-alerts',
        Subject=f'ECS Service Restarted: {service}',
        Message=f'Service {service} in cluster {cluster} was restarted due to {alarm_name}'
    )

    return {
        'statusCode': 200,
        'body': json.dumps('Remediation completed')
    }
```

5.3 Incident Response Workflows

Systems Manager OpsCenter

Purpose: Centralized view of operational issues

OpsItem Sources:

- CloudWatch alarms
- EventBridge events
- AWS Security Hub
- AWS Config
- Manual creation

OpsItem Attributes:

- Title and description
- Priority (1-5)
- Status (Open, In progress, Resolved)
- Related resources
- Runbooks
- SNS notifications

Automated OpsItem Creation:

```
bash

# Create OpsItem from CloudWatch alarm
aws ssm create-ops-item \
  --title "High CPU on Production Instance" \
  --description "CPU utilization exceeded 80% for 10 minutes" \
  --priority 2 \
  --source "cloudwatch-alarm" \
  --operational-data '{
    "AlarmName": {"Value": "HighCPU", "Type": "SearchableString"},
    "InstanceId": {"Value": "i-1234567890abcdef0", "Type": "SearchableString"}
  }' \
  --notifications Arn=arn:aws:sns:us-east-1:123456789012:ops-team
```

Incident Manager (AWS Systems Manager)

Purpose: Manage and respond to incidents

Key Features:

- Incident response plans
- Runbooks for automated remediation
- Contact routing
- Engagement escalation
- Post-incident analysis
- Chat channel integration (Slack, Chime)

Response Plan Example:

yaml

Title: High Error Rate Response

IncidentTemplate:

Title: High Error Rate Detected

Impact: 3

Summary: Application error rate exceeded threshold

DedupeString: high-error-rate

Engagements:

- **ContactArn:** arn:aws:ssm-contacts:us-east-1:123456789012:contact/on-call-engineer

Actions:

- **SsmAutomation:**

RoleArn: arn:aws:iam::123456789012:role/IncidentManager-Automation

DocumentName: AWS-RestartEC2Instance

Parameters:

InstanceId:

- "\${INCIDENT_RESOURCE_ARN}"

ChatChannel:

ChatbotSns:

- arn:aws:sns:us-east-1:123456789012:incident-chat

Automated Runbooks

Common Patterns:

1. Auto-Healing:

yaml

Detect → Diagnose → Remediate → Verify

- Detect unhealthy instance (CloudWatch/ELB)

- Run diagnostic commands

- Attempt restart

- If failed, replace instance

- Verify health check passes

- Notify team

2. Traffic Shifting:

yaml

High error rate response

- Detect elevated errors (CloudWatch)
- Shift traffic to known-good version (ALB/Route53)
- Investigate root cause
- Notify on-call team

3. Capacity Management:

yaml

Scale based on load

- Detect high utilization (CloudWatch)
- Check current capacity
- Calculate required capacity
- Trigger Auto Scaling
- Monitor stabilization
- Send summary report

5.4 Troubleshooting and Root Cause Analysis

CloudWatch Logs Insights for Troubleshooting

Common Queries:

Find Correlated Errors:

```
fields @timestamp, @message, @requestId
| filter @message like /ERROR|FATAL/
| stats count() by @requestId
| sort count desc
```

Identify Error Patterns:

```
fields @timestamp, @message
| parse @message /Error: (?<errorType>.*)/
| stats count() by errorType
| sort count desc
```

Track Request Flow:

```
fields @timestamp, @message  
| filter @requestId = "abc-123-def"  
| sort @timestamp asc
```

X-Ray for Root Cause Analysis

Identify Bottlenecks:

1. View service map
2. Sort by latency
3. Examine slow segments
4. Check subsegments for external calls

Analyze Error Distribution:

```
fault = true AND service("api-service")
```

Compare Time Periods:

- Before and after deployment
- Peak vs off-peak hours
- Different regions

AWS Fault Injection Simulator (FIS)

Purpose: Chaos engineering for resilience testing

Supported Actions:

- Stop EC2 instances
- Throttle API calls
- Inject CPU stress
- Network latency
- I/O stress
- ECS task termination

Example Experiment:

json

```
{
  "description": "Test application resilience to instance failures",
  "targets": {
    "myInstances": {
      "resourceType": "aws:ec2:instance",
      "resourceTags": {
        "Environment": "staging"
      },
      "selectionMode": "COUNT(2)"
    }
  },
  "actions": {
    "stopInstances": {
      "actionId": "aws:ec2:stop-instances",
      "parameters": {
        "duration": "PT5M"
      },
      "targets": {
        "Instances": "myInstances"
      }
    }
  },
  "stopConditions": [
    {
      "source": "aws:cloudwatch:alarm",
      "value": "arn:aws:cloudwatch:us-east-1:123456789012:alarm:CriticalAlarm"
    }
  ]
}
```

Best Practices:

- Start with non-production environments
- Use stop conditions to limit blast radius
- Run during business hours initially
- Document findings and improvements
- Automate regular chaos experiments

Domain 6: Security and Compliance (17%)

6.1 IAM Best Practices

Principle of Least Privilege

Service Control Policies (SCPs):

json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": [
        "ec2:RunInstances"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "ec2:InstanceType": [
            "t3.micro",
            "t3.small",
            "t3.medium"
          ]
        }
      }
    }
  ]
}
```

IAM Policy Example:

json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3::my-bucket/users/${aws:username}/*"
    },
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3::my-bucket",
      "Condition": {
        "StringLike": {
          "s3:prefix": "users/${aws:username}/*"
        }
      }
    }
  ]
}
```

IAM Roles for Cross-Account Access

Trust Policy (in Account B):

json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::ACCOUNT-A:root"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "unique-external-id"
        }
      }
    }
  ]
}
```

Assume Role (from Account A):

python

```
import boto3

sts = boto3.client('sts')

assumed_role = sts.assume_role(
    RoleArn='arn:aws:iam::ACCOUNT-B:role/CrossAccountRole',
    RoleSessionName='DevOpsSession',
    ExternalId='unique-external-id'
)

credentials = assumed_role['Credentials']

# Use temporary credentials
s3 = boto3.client(
    's3',
    aws_access_key_id=credentials['AccessKeyId'],
    aws_secret_access_key=credentials['SecretAccessKey'],
    aws_session_token=credentials['SessionToken']
)
```

IAM Identity Center (AWS SSO)

Purpose: Centralized access management for multiple AWS accounts

Features:

- Single sign-on to AWS accounts
- Integration with identity providers (AD, Okta, etc.)
- Permission sets for consistent access
- MFA enforcement
- Audit logging

Permission Set Example:

```
json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:Describe*",
        "cloudwatch:Get*",
        "cloudwatch:List*"
      ],
      "Resource": "*"
    }
  ]
}
```

IAM Access Analyzer

Purpose: Identify resources shared with external entities

Findings:

- S3 buckets with public access
- IAM roles assumable by external accounts
- KMS keys shared externally
- Lambda functions with external permissions
- SQS queues with external access

Policy Validation:

- Check policy for errors and warnings
 - Validate against policy grammar
 - Security best practice recommendations
-

6.2 Secrets Management

AWS Secrets Manager

Purpose: Securely store and automatically rotate secrets

Features:

- Automatic rotation (Lambda-based)
- Encryption at rest (KMS)
- Fine-grained access control
- Audit via CloudTrail
- Versioning

Create Secret:

```
bash

aws secretsmanager create-secret \
  --name prod/database/credentials \
  --description "Production database credentials" \
  --secret-string '{
    "username": "admin",
    "password": "MySecurePassword123!",
    "host": "db.example.com",
    "port": 5432
  }'
```

Retrieve Secret:

```
python
```

```
import boto3
```

```
import json
```

```
secretsmanager = boto3.client('secretsmanager')
```

```
response = secretsmanager.get_secret_value(SecretId='prod/database/credentials')
```

```
secret = json.loads(response['SecretString'])
```

```
username = secret['username']
```

```
password = secret['password']
```

Rotation Configuration:

```
bash
```

```
aws secretsmanager rotate-secret \
```

```
--secret-id prod/database/credentials \
```

```
--rotation-lambda-arn arn:aws:lambda:us-east-1:123456789012:function:SecretsManagerRotation
```

```
--rotation-rules AutomaticallyAfterDays=30
```

Automatic Rotation Lambda:

- RDS/Aurora: Built-in rotation templates
- Other databases: Custom Lambda function
- Four rotation steps: createSecret, setSecret, testSecret, finishSecret

Parameter Store (Advanced Parameters)

When to Use Parameter Store vs Secrets Manager:

Feature	Parameter Store	Secrets Manager
Cost	Free (Standard), \$0.05/param/month (Advanced)	\$0.40/secret/month + API calls
Rotation	Manual	Automatic
Size Limit	4KB (Standard), 8KB (Advanced)	10KB
Use Case	Configuration, non-sensitive	Secrets, credentials
Versioning	Limited	Full

6.3 Encryption

Encryption at Rest

S3 Encryption Options:

Type	Description	Use Case
SSE-S3	S3-managed keys (AES-256)	Default, simple
SSE-KMS	KMS-managed keys	Audit, rotation, access control
SSE-C	Customer-provided keys	Full key control
Client-side	Encrypt before upload	Maximum security

EBS Encryption:

- Encryption at rest using KMS
- Encryption of data in transit
- Encrypted snapshots
- Minimal performance impact

RDS Encryption:

- Encrypt new databases at creation (cannot encrypt existing)
- Encrypted snapshots and read replicas
- Transparent to applications

DynamoDB Encryption:

- Encryption at rest enabled by default
- AWS-owned, AWS-managed, or customer-managed KMS keys
- No performance impact

Encryption in Transit

Best Practices:

- Use TLS/SSL for all connections
- Enforce HTTPS for S3 buckets
- Use VPN or Direct Connect for hybrid connectivity
- Enable TLS for RDS connections
- Use HTTPS endpoints for APIs

S3 Bucket Policy (Enforce HTTPS):

json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::my-bucket",
        "arn:aws:s3:::my-bucket/*"
      ],
      "Condition": {
        "Bool": {
          "aws:SecureTransport": "false"
        }
      }
    }
  ]
}
```

AWS Key Management Service (KMS)

Key Types:

Type	Description	Use Case
AWS-managed	Created and managed by AWS	Default, simple
Customer-managed	Full control, rotation	Compliance, audit
AWS-owned	Used by AWS services	Transparent to user

Key Policy Example:

json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Enable IAM User Permissions",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:root"
      },
      "Action": "kms:*",
      "Resource": "*"
    },
    {
      "Sid": "Allow use of key",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:role/AppRole"
      },
      "Action": [
        "kms:Decrypt",
        "kms:DescribeKey"
      ],
      "Resource": "*"
    }
  ]
}
```

Automatic Key Rotation:

- Enabled for customer-managed keys
- Rotates annually
- Previous key versions retained for decryption
- Manual rotation for imported keys

Envelope Encryption:

Data → Encrypt with Data Key → Encrypted Data
Data Key → Encrypt with KMS Key → Encrypted Data Key

Store: Encrypted Data + Encrypted Data Key

Decrypt:

Encrypted Data Key → Decrypt with KMS Key → Data Key

Encrypted Data → Decrypt with Data Key → Data

6.4 Compliance and Governance

AWS Config

Purpose: Track resource configurations and compliance

Config Rules Examples:

Rule	Purpose
encrypted-volumes	Ensure EBS volumes are encrypted
s3-bucket-public-read-prohibited	No public read access
iam-password-policy	Enforce password requirements
cloud-trail-enabled	CloudTrail logging enabled
vpc-flow-logs-enabled	VPC Flow Logs enabled
rds-multi-az-support	RDS instances use Multi-AZ

Custom Config Rule (Lambda):

python

```
import boto3
import json

config = boto3.client('config')

def lambda_handler(event, context):
    # Get resource configuration
    configuration_item = json.loads(event['configurationItem'])

    # Check compliance
    compliance_status = 'COMPLIANT'
    if configuration_item['resourceType'] == 'AWS::S3::Bucket':
        # Check if versioning is enabled
        s3 = boto3.client('s3')
        versioning = s3.get_bucket_versioning(
            Bucket=configuration_item['resourceName']
        )
        if versioning.get('Status') != 'Enabled':
            compliance_status = 'NON_COMPLIANT'

    # Put evaluation
    config.put_evaluations(
        Evaluations=[
            {
                'ComplisonResourceType': configuration_item['resourceType'],
                'ComplianceResourceId': configuration_item['resourceId'],
                'ComplianceType': compliance_status,
                'OrderingTimestamp': configuration_item['configurationItemCaptureTime']
            }
        ],
        ResultToken=event['resultToken']
    )
```

Config Aggregator:

- Multi-account, multi-region view
- Centralized compliance dashboard
- Organization-wide governance

Conformance Packs:

- Collection of Config rules
- Pre-built packs for compliance frameworks
- Deploy across Organization

AWS CloudTrail

Purpose: API activity logging and auditing

Best Practices:

- Enable in all regions
- Enable log file integrity validation
- Store logs in S3 with Object Lock
- Use CloudWatch Logs for real-time monitoring
- Enable CloudTrail Insights for anomaly detection

CloudTrail Configuration:

```
bash
```

```
aws cloudtrail create-trail \  
  --name organization-trail \  
  --s3-bucket-name cloudtrail-logs-bucket \  
  --include-global-service-events \  
  --is-multi-region-trail \  
  --enable-log-file-validation \  
  --cloud-watch-logs-log-group-arn arn:aws:logs:us-east-1:123456789012:log-group:cloudtrail-logs \  
  --cloud-watch-logs-role-arn arn:aws:iam::123456789012:role/CloudTrailToCloudWatchLogs
```

CloudTrail Insights:

- Detect unusual API activity
- Identify write management events
- Anomaly detection via ML
- Automatic alerts

Example Insights:

- Unusual number of EC2 `RunInstances` calls
- Spike in IAM policy changes
- Abnormal S3 deletion activity

AWS Organizations

Purpose: Centrally manage multiple AWS accounts

Organizational Units (OUs):

```
Root
├─ Production OU
│   └─ Account A (Web App)
│       └─ Account B (Database)
├─ Development OU
│   └─ Account C (Dev Environment)
│       └─ Account D (Test Environment)
└─ Security OU
    └─ Account E (Security Tools)
```

Service Control Policies (SCPs):

- Apply at OU or account level
- Override IAM permissions (guardrails)
- Deny List vs Allow List strategies

Example SCP (Prevent Region Usage):

```
json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestedRegion": [
            "us-east-1",
            "us-west-2"
          ]
        }
      }
    }
  ]
}
```

Consolidated Billing:

- Single payment method
- Volume discounts
- Reserved Instance sharing
- Cost allocation tags

AWS Control Tower

Purpose: Set up and govern multi-account AWS environment

Features:

- Account factory for provisioning
- Guardrails (SCPs and Config rules)
- Dashboard for compliance
- Account baseline (IAM Identity Center, CloudTrail, Config)

Guardrails:

- **Mandatory:** Always enforced
- **Strongly recommended:** Best practices
- **Elective:** Optional based on needs

Example Guardrails:

- Disallow public read access to S3 buckets
- Detect unencrypted EBS volumes
- Enable MFA for root account
- Disallow internet access without VPN

AWS Audit Manager

Purpose: Automated evidence collection for compliance audits

Supported Frameworks:

- SOC 2
- PCI DSS
- GDPR
- HIPAA
- Custom frameworks

Features:

- Continuous evidence collection
- Pre-built frameworks
- Automated assessment reports
- Audit-ready compliance reports

6.5 Network Security

Security Groups vs NACLs

Feature	Security Group	Network ACL
Level	Instance	Subnet
State	Stateful	Stateless
Rules	Allow only	Allow and Deny
Evaluation	All rules	Rules in order
Assignment	Multiple per instance	One per subnet
Return Traffic	Automatic	Must be explicitly allowed

Security Group Example:

```
bash

aws ec2 authorize-security-group-ingress \
  --group-id sg-1234567890abcdef0 \
  --protocol tcp \
  --port 443 \
  --cidr 0.0.0.0/0

aws ec2 authorize-security-group-ingress \
  --group-id sg-1234567890abcdef0 \
  --protocol tcp \
  --port 22 \
  --source-group sg-0987654321fedcba0
```

NACL Example:

bash

Allow HTTPS inbound

```
aws ec2 create-network-acl-entry \  
  --network-acl-id acl-1234567890abcdef0 \  
  --rule-number 100 \  
  --protocol tcp \  
  --port-range From=443,To=443 \  
  --cidr-block 0.0.0.0/0 \  
  --ingress \  
  --rule-action allow
```

Allow return traffic (ephemeral ports)

```
aws ec2 create-network-acl-entry \  
  --network-acl-id acl-1234567890abcdef0 \  
  --rule-number 100 \  
  --protocol tcp \  
  --port-range From=1024,To=65535 \  
  --cidr-block 0.0.0.0/0 \  
  --egress \  
  --rule-action allow
```

AWS WAF Rules

Rule Type	Description
Managed Rules	Pre-configured by AWS or partners
Rate-based	Block IPs exceeding threshold
IP Match	Allow/block specific IPs
Geo Match	Filter by country
String Match	Check request content
Regex Match	Pattern matching

WAF Web ACL Example:

json

```
{
  "Name": "ProductionWebACL",
  "Rules": [
    {
      "Name": "AWSManagedRulesCommonRuleSet",
      "Priority": 1,
      "Statement": {
        "ManagedRuleGroupStatement": {
          "VendorName": "AWS",
          "Name": "AWSManagedRulesCommonRuleSet"
        }
      },
      "OverrideAction": {"None": {}},
      "VisibilityConfig": {
        "SampledRequestsEnabled": true,
        "CloudWatchMetricsEnabled": true,
        "MetricName": "CommonRuleSetMetric"
      }
    },
    {
      "Name": "RateLimitRule",
      "Priority": 2,
      "Statement": {
        "RateBasedStatement": {
          "Limit": 2000,
          "AggregateKeyType": "IP"
        }
      },
      "Action": {"Block": {}},
      "VisibilityConfig": {
        "SampledRequestsEnabled": true,
        "CloudWatchMetricsEnabled": true,
        "MetricName": "RateLimitMetric"
      }
    }
  ],
  "DefaultAction": {"Allow": {}},
  "VisibilityConfig": {
    "SampledRequestsEnabled": true,
    "CloudWatchMetricsEnabled": true,
    "MetricName": "ProductionWebACL"
  }
}
```

AWS Shield:

- **Standard:** Free DDoS protection (automatic)
 - **Advanced:** Enhanced DDoS protection, 24/7 DRT support, cost protection
-

6.6 Security Monitoring and Auditing

Amazon GuardDuty

Finding Type	Source
EC2	VPC Flow Logs, DNS logs
IAM	CloudTrail management events
S3	CloudTrail S3 data events
EKS	EKS audit logs
RDS	RDS login activity
Lambda	Network activity
Runtime	Runtime Monitoring for EC2, ECS, EKS

Finding Severity:

- **High:** Immediate attention (8.0-10.0)
- **Medium:** Investigate promptly (4.0-7.9)
- **Low:** Informational (0.1-3.9)

Example Findings:

- UnauthorizedAccess:EC2/SSHBruteForce
- Recon:EC2/PortProbeUnprotectedPort
- CryptoCurrency:EC2/BitcoinTool.B!DNS
- Stealth:S3/ServerAccessLoggingDisabled

Automated Response:

python

```
import boto3
```

```
guardduty = boto3.client('guardduty')
```

```
sns = boto3.client('sns')
```

```
ec2 = boto3.client('ec2')
```

```
def lambda_handler(event, context):
```

```
    finding = event['detail']
```

```
    severity = finding['severity']
```

```
    if severity >= 7.0:
```

```
        # High severity - isolate instance
```

```
        instance_id = finding['resource']['instanceDetails']['instanceId']
```

```
        # Attach isolation security group
```

```
        ec2.modify_instance_attribute(
```

```
            InstanceId=instance_id,
```

```
            Groups=['sg-isolation']
```

```
        )
```

```
        # Notify security team
```

```
        sns.publish(
```

```
            TopicArn='arn:aws:sns:us-east-1:123456789012:security-alerts',
```

```
            Subject=f'GuardDuty High Severity Finding',
```

```
            Message=f'Instance {instance_id} isolated due to security finding'
```

```
        )
```

Amazon Inspector

Purpose: Automated vulnerability management

Assessments:

- EC2 instances: OS vulnerabilities, network reachability
- ECR images: Container vulnerabilities
- Lambda functions: Code vulnerabilities

Findings:

- CVE-based vulnerabilities
- Network exposure
- CIS benchmarks
- Security best practices

Integration:

- Security Hub for centralized findings
- EventBridge for automated workflows
- Systems Manager for remediation

AWS Security Hub

Purpose: Centralized security findings aggregation

Features:

- Aggregate findings from GuardDuty, Inspector, Macie, etc.
- Compliance checks against standards
- Custom insights and dashboards
- Automated response workflows

Compliance Standards:

Standard	Description
AWS Foundational Security Best Practices	AWS security controls
CIS AWS Foundations	Center for Internet Security benchmarks
PCI DSS	Payment card industry requirements
NIST 800-53	Federal security controls

Custom Actions:

python

```
def lambda_handler(event, context):
    finding = event['detail']['findings'][0]

    # Extract relevant info
    severity = finding['Severity']['Label']
    resource_id = finding['Resources'][0]['Id']

    if severity == 'CRITICAL':
        # Create Jira ticket
        create_jira_ticket(finding)

        # Notify security team
        notify_team(finding)

        # Start remediation runbook
        start_runbook(resource_id)
```

Amazon Macie

Purpose: Discover and protect sensitive data in S3

Capabilities:

- Identify PII, financial data, credentials
- Evaluate S3 bucket security posture
- Generate findings for sensitive data

Sensitive Data Types:

- Personally Identifiable Information (PII)
- Financial information (credit cards)
- Credentials (API keys, passwords)
- Healthcare data
- Custom data identifiers (regex-based)

Automated Classification:

- Continuous monitoring
- ML-based detection
- Custom data identifier patterns

VPC Flow Logs

Capture: Source/dest IP, ports, protocol, action, bytes

Destinations: CloudWatch Logs, S3, Kinesis Data Firehose

Levels: VPC, Subnet, ENI

Log Format:

```
version account-id interface-id srcaddr dstaddr srcport dstport protocol packets bytes start end  
action log-status
```

Analysis with Athena:

sql

```
CREATE EXTERNAL TABLE IF NOT EXISTS vpc_flow_logs (  
  version int,  
  account_id string,  
  interface_id string,  
  srcaddr string,  
  dstaddr string,  
  srcport int,  
  dstport int,  
  protocol int,  
  packets bigint,  
  bytes bigint,  
  start_time bigint,  
  end_time bigint,  
  action string,  
  log_status string  
)  
PARTITIONED BY (dt string)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ' '  
LOCATION 's3://my-vpc-flow-logs/'
```

-- Find top talkers

```
SELECT srcaddr, dstaddr, SUM(bytes) as total_bytes  
FROM vpc_flow_logs  
WHERE dt = '2024-12-04'  
GROUP BY srcaddr, dstaddr  
ORDER BY total_bytes DESC  
LIMIT 10;
```

-- Find rejected connections

```
SELECT srcaddr, dstaddr, srcport, dstport, COUNT(*) as count  
FROM vpc_flow_logs  
WHERE action = 'REJECT' AND dt = '2024-12-04'  
GROUP BY srcaddr, dstaddr, srcport, dstport  
ORDER BY count DESC;
```

CloudTrail for Security Auditing

Best Practices:

- Enable for all regions
- Enable log file integrity validation
- Store in S3 with Object Lock
- Monitor with CloudWatch Logs metric filters
- Use CloudTrail Insights for anomaly detection

Security-Focused Metric Filters:

Root Account Usage:

```
{ $.userIdentity.type = "Root" && $.userIdentity.invokedBy NOT EXISTS && $.eventType != "AwsServiceEvent" }
```

Unauthorized API Calls:

```
{ $.errorCode = "*UnauthorizedOperation" || $.errorCode = "AccessDenied*" }
```

IAM Policy Changes:

```
{ $.eventName = "PutUserPolicy" || $.eventName = "PutRolePolicy" || $.eventName = "PutGroupPolicy" || $.eventName = "CreatePolicy*" || $.eventName = "DeletePolicy*" || $.eventName = "AttachUserPolicy" || $.eventName = "DetachUserPolicy" }
```

Security Group Changes:

```
{ $.eventName = "AuthorizeSecurityGroupIngress" || $.eventName = "AuthorizeSecurityGroupEgress" || $.eventName = "RevokeSecurityGroupIngress" || $.eventName = "RevokeSecurityGroupEgress" }
```

CloudTrail Insights:

```
bash

aws cloudtrail put-insight-selectors \
  --trail-name my-trail \
  --insight-selectors '[{"InsightType": "ApiCallRateInsight"}]'
```

AWS Services Quick Reference

Developer Tools

Service	Purpose
CodePipeline	CI/CD orchestration
CodeBuild	Build and test
CodeDeploy	Deployment automation
CodeArtifact	Artifact repository
X-Ray	Distributed tracing
CodeGuru	ML-powered code reviews
Fault Injection Simulator	Chaos engineering

Management & Governance

Service	Purpose
CloudFormation	Infrastructure as Code
CloudWatch	Monitoring and observability
Config	Configuration compliance
Systems Manager	Operations management
Control Tower	Multi-account governance
Organizations	Account management
Service Catalog	Self-service provisioning
Trusted Advisor	Best practice recommendations
OpsWorks	Configuration management (Chef/Puppet)

Compute

Service	Purpose
EC2	Virtual servers
Lambda	Serverless compute
ECS	Container orchestration
EKS	Kubernetes
Fargate	Serverless containers
Elastic Beanstalk	PaaS deployment
App Runner	Container web apps

Networking

Service	Purpose
Route 53	DNS and routing
CloudFront	CDN
ELB (ALB/NLB/GWLB)	Load balancing
API Gateway	API management
VPC	Virtual network
PrivateLink	Private connectivity
Global Accelerator	Network optimization

Security

Service	Purpose
IAM	Identity and access
IAM Identity Center	SSO
Secrets Manager	Secrets management
KMS	Key management
ACM	Certificate management
WAF	Web application firewall
Shield	DDoS protection
GuardDuty	Threat detection
Inspector	Vulnerability assessment
Security Hub	Security posture
Macie	Data discovery
Detective	Security investigation

Storage

Service	Purpose
S3	Object storage
EBS	Block storage
EFS	File storage
FSx	Windows/Lustre file systems
Backup	Centralized backup

Database

Service	Purpose
RDS	Relational databases
Aurora	MySQL/PostgreSQL compatible
DynamoDB	NoSQL
ElastiCache	In-memory caching

Application Integration

Service	Purpose
EventBridge	Event bus
SNS	Pub/sub messaging
SQS	Message queuing
Step Functions	Workflow orchestration
AppSync	GraphQL API

Analytics

Service	Purpose
Kinesis	Real-time streaming
Athena	S3 query
OpenSearch	Search and analytics
QuickSight	Business intelligence

Exam Tips

Common Exam Scenarios

Scenario	Likely Solution
Deploy across multiple accounts/regions	CloudFormation StackSets
Automate compliance remediation	Config Rules + SSM Automation
Blue/green deployment for Lambda	CodeDeploy with aliases
Secrets with automatic rotation	Secrets Manager
Configuration without rotation	Parameter Store
Cross-account access	IAM roles with trust policies
Centralized logging	CloudWatch Logs + Kinesis Firehose
Deployment rollback	CodeDeploy automatic rollback on failure
Infrastructure drift detection	CloudFormation drift detection, Config
Event-driven automation	EventBridge → Lambda/Step Functions
Container deployment	ECS/EKS with CodeDeploy
API versioning with traffic shift	API Gateway canary release
Disaster recovery with minimal RTO	Warm standby or multi-site
Cost-effective DR	Pilot light
Workflow orchestration	Step Functions
Microservices coordination	Step Functions + EventBridge

Key Differentiators to Remember

CloudFormation vs CDK vs SAM

- **CloudFormation:** JSON/YAML templates, direct AWS integration
- **CDK:** Programming languages, generates CloudFormation
- **SAM:** Serverless-focused, extends CloudFormation

Parameter Store vs Secrets Manager

- **Parameter Store:** Free tier, configuration data, manual rotation
- **Secrets Manager:** Paid, automatic rotation, database credentials

CodeDeploy Deployment Types by Platform

- **EC2/On-premises:** In-place, Blue/Green
- **Lambda:** Canary, Linear, All-at-once
- **ECS:** Blue/Green only

Blue/Green vs Canary

- **Blue/Green:** Full environment switch, instant rollback
- **Canary:** Gradual traffic shift, validate before full rollout

GuardDuty vs Inspector vs Macie

- **GuardDuty:** Threat detection across AWS
- **Inspector:** Vulnerability scanning for EC2, ECR, Lambda
- **Macie:** Sensitive data discovery in S3

Standard vs Express Step Functions

- **Standard:** Long-running (up to 1 year), exactly-once execution
- **Express:** Short-duration (up to 5 minutes), at-least-once, cheaper for high volume

CloudWatch Events vs EventBridge

- **CloudWatch Events:** Legacy, limited features
 - **EventBridge:** Preferred, multiple event buses, SaaS integration
-

Study Checklist

- ☐ Understand all six exam domains and their weightings
 - ☐ Practice with AWS free tier for hands-on experience
 - ☐ Review official AWS documentation for each service
 - ☐ Complete practice exams from AWS Skill Builder
 - ☐ Understand deployment strategies (blue/green, canary, rolling)
 - ☐ Know when to use each IaC tool (CloudFormation, CDK, SAM)
 - ☐ Master CloudWatch metrics, logs, alarms, and dashboards
 - ☐ Understand X-Ray for distributed tracing
 - ☐ Know EventBridge patterns for event-driven architecture
 - ☐ Practice IAM policies and cross-account scenarios
 - ☐ Understand multi-account governance with Organizations and Control Tower
 - ☐ Review disaster recovery strategies and their trade-offs
 - ☐ Know Auto Scaling policies and when to use each
 - ☐ Understand security services (GuardDuty, Security Hub, Inspector)
 - ☐ Practice troubleshooting deployment failures
 - ☐ Understand Step Functions state types and orchestration patterns
 - ☐ Know Systems Manager capabilities (Parameter Store, Session Manager, Automation)
 - ☐ Review compliance frameworks and Config rules
 - ☐ Understand secrets management and rotation strategies
 - ☐ Practice writing CloudWatch Logs Insights queries
-

Official Resources:

- AWS Certified DevOps Engineer - Professional Exam Guide: https://d1.awsstatic.com/training-and-certification/docs-devops-pro/AWS-Certified-DevOps-Engineer-Professional_Exam-Guide.pdf
 - AWS Documentation: <https://docs.aws.amazon.com/>
 - AWS Skill Builder: <https://skillbuilder.aws/>
 - AWS Well-Architected Framework: <https://aws.amazon.com/architecture/well-architected/>
-

Last Updated: December 2024

Exam Version: DOP-C02

Author: Matrix Agent